

week4

학습 목표

1. **Reconcile 함수의 표준 구조** — 어떤 순서로 뭘 해야 하는지
2. **CRUD 패턴** — 자식 리소스를 Get → 없으면 Create, 있으면 Update
3. **OwnerReference** — 부모 CR이 삭제되면 자식도 자동 삭제되도록 묶기

1. Reconcile 함수의 표준 구조

1.1. 시그니처와 입력

```
func (r *MemoReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    // ...
}
```

- `ctx`: 컨텍스트. 타임아웃/취소 신호 전달용. 모든 client 호출에 그대로 넘김.
- `req`: `req.NamespacedName` 만 들어있음 (`namespace/name`). **CR의 내용은 안 들어옴.**
- 반환값 `ctrl.Result`: 재호출 스케줄링 지시
- 반환값 `error`: nil이 아니면 controller-runtime이 rate-limited 재시도

왜 `req`에는 이름만 오는가?: 이벤트가 큐에 쌓이고 워커가 꺼낼 때까지 시간이 흐를 수 있어서, 그 사이에 리소스가 바뀌었을 수도 있습니다. 그래서 항상 "이름만 받고, 최신 상태는 **Reconcile 안에서 다시 Get**"하는 게 원칙입니다.

1.2. 표준 흐름 (7단계)

거의 모든 오퍼레이터의 `Reconcile`이 따르는 패턴

```
func (r *MemoReconciler) Reconcile(ctx context.Context, req ctrl.Request)
(ctrl.Result, error) {
    log := log.FromContext(ctx)

    // 1. 부모 CR 가져오기
    var memo blogv1alpha1.Memo
    if err := r.Get(ctx, req.NamespacedName, &memo); err != nil {
        if apierrors.IsNotFound(err) {
            // 이미 삭제됨 - 자식은 OwnerReference로 GC가 처리. 그냥 끝.
            return ctrl.Result{}, nil
        }
        return ctrl.Result{}, err
    }

    // 2. 삭제 처리 (Finalizer 쓸 때만 - 이번 주차에선 생략)
    // if !memo.DeletionTimestamp.IsZero() { ... }

    // 3. Desired State 계산 - Spec을 기반으로 "있어야 할" 자식 리소스 정의
```

```

desired := buildConfigMap(&memo)

// 4. OwnerReference 설정 - "이 자식은 memo가 부모다"
if err := ctrl.SetControllerReference(&memo, desired, r.Scheme); err != nil {
    return ctrl.Result{}, err
}

// 5. 자식 리소스 reconcile - 없으면 만들고, 있으면 차이를 맞춤
var existing corev1.ConfigMap
err := r.Get(ctx, client.ObjectKeyFromObject(desired), &existing)
switch {
case apierrors.IsNotFound(err):
    if err := r.Create(ctx, desired); err != nil {
        return ctrl.Result{}, err
    }
    log.Info("created ConfigMap", "name", desired.Name)
case err != nil:
    return ctrl.Result{}, err
default:
    // 이미 존재 - Spec이 다르면 Update
    if !reflect.DeepEqual(existing.Data, desired.Data) {
        existing.Data = desired.Data
        if err := r.Update(ctx, &existing); err != nil {
            return ctrl.Result{}, err
        }
        log.Info("updated ConfigMap", "name", existing.Name)
    }
}

// 6. Status 업데이트 - 관찰된 현재 상태를 CR에 기록
memo.Status.LastUpdated = metav1.Now()
if err := r.Status().Update(ctx, &memo); err != nil {
    return ctrl.Result{}, err
}

// 7. 반환
return ctrl.Result{}, nil
}

```

1.3. ctrl.Result 반환값

반환값	의미
<code>ctrl.Result{}, nil</code>	정상 종료. 다음 Watch 이벤트까지 호출 안 함.
<code>ctrl.Result{}, err</code>	에러. rate-limited backoff 후 재시도 (수 초 → 수 분).
<code>ctrl.Result{Requeue: true}, nil</code>	즉시 큐에 다시 넣음.
<code>ctrl.Result{RequeueAfter: 30*time.Second}, nil</code>	30초 후 재호출. 외부 API 폴링이나 타이머 기반 작업에 사용.

중요: 정상이면 `ctrl.Result{}`, `nil` 반환하고 끝.

일부러 주기적으로 호출되도록 짜면 X — 컨트롤러 패턴은 **Watch 기반**이지 폴링 X

2. 리소스 CRUD 로직

2.1. controller-runtime의 client.Client

`r.Client` 는 다음 메서드를 제공합니다.

```
r.Get(ctx, key, obj)           // 단건 조회
r.List(ctx, list, opts...)     // 다건 조회
r.Create(ctx, obj)             // 생성
r.Update(ctx, obj)             // 전체 업데이트
r.Patch(ctx, obj, patch)       // 부분 업데이트
r.Delete(ctx, obj)             // 삭제
r.Status().Update(ctx, obj)     // status 서브리소스만 업데이트
r.Status().Patch(ctx, obj, p)  // status 서브리소스 patch
```

`Get` 과 `List` 는 **로컬 캐시**에서 읽습니다 (Informer가 채워둔 거). API 서버로 직접 가지 않아요. 그래서 빠르고 부하가 없습니다. `Create/Update/Delete/Patch` 는 API 서버로 직접 갑니다.

2.2. Get-or-Create 패턴

가장 자주 쓰는 패턴 (5단계에 해당)

```
var cm corev1.ConfigMap
err := r.Get(ctx, types.NamespacedName{Name: name, Namespace: ns}, &cm)
if apierrors.IsNotFound(err) {
    // 없음 → 생성
    cm = buildConfigMap(...)
    return r.Create(ctx, &cm)
}
if err != nil {
    return err // 진짜 에러
}
// 있음 → 차이 비교 후 필요시 Update
```

`apierrors.IsNotFound(err)` 이 핵심입니다. "NotFound"는 에러처럼 보이지만 **정상 흐름의 일부**예요.

2.3. CreateOrUpdate 헬퍼

controller-runtime이 위 패턴을 한 줄로 줄여주는 헬퍼를 제공

```
import "sigs.k8s.io/controller-runtime/pkg/controller/controllerutil"

cm := &corev1.ConfigMap{
    ObjectMeta: metav1.ObjectMeta{
        Name:      memo.Name,
        Namespace: memo.Namespace,
    },
}
```

```

}

op, err := controllerutil.CreateOrUpdate(ctx, r.Client, cm, func() error {
    // mutate function - 원하는 상태로 cm을 변경
    cm.Data = map[string]string{
        "message": memo.Spec.Message,
    }
    return ctrl.SetControllerReference(&memo, cm, r.Scheme)
})
if err != nil {
    return ctrl.Result{}, err
}
log.Info("reconciled", "op", op) // op = "created" | "updated" | "unchanged"

```

2.4. Update vs Patch

	Update	Patch
동작	객체 전체를 교체	일부 필드만 변경
Conflict	resourceVersion 안 맞으면 실패	보통 안전
사용 시점	내가 Get한 그대로 수정해서 다시 보낼 때	특정 필드만 살짝 바꿀 때

처음엔 **Update**만 써도 충분합니다. Get → 수정 → Update.

2.5. Spec과 Status는 분리해서 업데이트

쿠버네티스의 관례입니다.

```

// ❌ 같이 업데이트 안 됨
memo.Spec.Foo = "x"
memo.Status.Bar = "y"
r.Update(ctx, &memo) // spec만 갱신되고 status는 무시될 수 있음

// ✅ 분리
memo.Spec.Foo = "x"
r.Update(ctx, &memo) // spec subresource

memo.Status.Bar = "y"
r.Status().Update(ctx, &memo) // status subresource

```

이게 분리된 이유: 컨트롤러는 Status만 쓰고, 사용자는 Spec만 쓰는 권한 분리를 가능하게 하기 위함. CRD 정의 시 `// +kubebuilder:subresource:status` 마커가 붙어 있으면 분리됩니다.

3. OwnerReference와 Ownership

3.1. 뭐고 왜 필요한가

OwnerReference는 "이 리소스는 저 리소스가 부모"라고 명시하는 메타데이터입니다.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: my-memo
  ownerReferences:
  - apiVersion: blog.abouthere.kr/v1alpha1
    kind: Memo
    name: my-memo
    uid: abc-123
    controller: true
    blockOwnerDeletion: true
data:
  message: hello

```

쿠버네티스의 **Garbage Collector**가 부모(Memo)가 삭제되면, 자식(ConfigMap)도 자동 삭제 를 해줍니다.

예시:

- Deployment → ReplicaSet → Pod (체인)
- Deployment 삭제하면 ReplicaSet도 Pod도 다 사라지는 그 동작이 OwnerReference 기반.

3.2. 코드로 설정하기

```

import "sigs.k8s.io/controller-runtime/pkg/controller/controllerutil"

// 자식 객체를 만든 직후, Create 호출 전에
if err := ctrl.SetControllerReference(&memo, cm, r.Scheme); err != nil {
    return ctrl.Result{}, err
}

```

`SetControllerReference` 가 자동으로:

- `controller: true` (이 부모가 주 컨트롤러임을 명시)
- `blockOwnerDeletion: true` (부모 GC 시 자식 정리 보장)
- 같은 네임스페이스 검증

주의사항:

- 부모와 자식은 **같은 네임스페이스**여야 함. 클러스터 스코프 리소스는 어느 네임스페이스의 자식도 가질 수 있지만, 그 반대는 안 됨.
- 한 자식에는 `controller: true` 인 OwnerReference가 **하나뿐**. 두 컨트롤러가 같은 자식을 두고 싸우면 안 되니까.

3.3. Watch 자동 등록

OwnerReference만큼 중요한 게 `SetupWithManager` 의 `Owns()`:

```

func (r *MemoReconciler) SetupWithManager(mgr ctrl.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&blogv1alpha1.Memo{}).

```

```

    Owns(&corev1.ConfigMap{}). // ← 이거
    Complete(r)
}

```

`Owns(&corev1.ConfigMap{})` 의 의미:

"내가 owner인 ConfigMap이 바뀌면, 그 owner(Memo)의 Reconcile을 다시 호출해줘"

→ 자식 리소스를 누가 손대도 부모의 **Reconcile**이 트리거되어 원상 복구됩니다.

`For()` 는 부모 리소스, `Owns()` 는 자식 리소스. 외부 리소스를 watch하려면 `Watches()` 도 있지만 이번 주차 범위 밖

5. 자주 하는 실수

실수	결과	해결
Reconcile 안에서 무한히 자식 생성	역등성 깨짐, ConfigMap 폭증	Get → 있으면 Update, 없으면 Create
OwnerReference 빼먹음	부모 삭제해도 자식 안 사라짐, orphan 발생	<code>SetControllerReference</code> 호출
<code>Owns()</code> 빼먹음	자식이 외부에서 수정돼도 reconcile 안 됨	<code>SetupWithManager</code> 에서 Owns 추가
Spec과 Status 같이 Update	Status 업데이트 안 됨	<code>r.Status().Update()</code> 분리
resourceVersion 안 맞아서 Conflict	<code>Operation cannot be fulfilled...</code> 에러	<code>ctrl.Result{}</code> , <code>err</code> 반환하면 controller-runtime이 재시도해줌
Get-Create 사이 race	동시에 두 번 만들려다 AlreadyExists 에러	Workqueue가 키 단위 직렬화하므로 같은 컨트롤러 내에선 발생 안 함. 다른 주체와 경쟁 시엔 <code>IsAlreadyExists</code> 처리
<code>r.Update(ctx, &memo)</code> 로 status 갱신 시도	spec subresource만 갱신, status 무시	<code>r.Status().Update()</code> 사

Keypoint

Spec 보고 자식 리소스 만들기, 자식엔 OwnerReference 붙이기, 변경 있을 때만 Update하기, Status는 따로 업데이트하기